



Universidade Federal do Rio Grande do Norte
Centro de Tecnologia - CT
Departamento de Engenharia Elétrica - DEE
Laboratório de Automação, Controle e Instrumentação - LACI

Máquina de Troco

ELE3515 - Grupo 01 - Problema 03 - Implementação

Líder:	Nicolas Pinheiro Silva
Redator:	Karen Helloisa Araújo Silva
Debatedor:	Rinaldo Tavares da Silva Filho

Natal, 21 de outubro de 2025

Resumo

Este trabalho apresenta o projeto e a implementação de uma máquina de troco inteligente na disciplina de Sistemas Digitais. O sistema foi desenvolvido para calcular trocos de forma automática, utilizando a metodologia de projeto RTL.

A implementação envolveu elementos combinacionais, como multiplexadores e demultiplexadores, e elementos de memória, como registradores, para armazenamento e processamento dos dados. A arquitetura foi organizada em módulos funcionais, incluindo registradores para o valor total da subtração, estado da máquina e quantidade de moedas em cada cofre. Cada módulo foi projetado, simulado e validado individualmente antes da integração ao sistema completo.

Os resultados confirmaram o funcionamento correto das operações propostas, evidenciando a viabilidade do projeto e proporcionando uma aplicação prática dos conceitos de circuitos digitais.

Palavras-chave: Máquina de troco; Sistemas Digitais; Projeto RTL.

Sumário

1	Introdução	5
2	Fundamentação Teórica	7
2.1	Metodologia de Projeto RTL	7
2.2	Demultiplexadores	9
3	Desenvolvimento da Solução	10
3.1	Botão Síncrono	10
3.2	Máquina de troco	12
3.2.1	Bloco Operacional	13
3.2.2	Bloco de Controle	16
3.3	Ligação do Operacional com o Controle	18
4	Implementação	21
4.1	Correção de projeto	21
4.2	Máquina de Estados	21
4.2.1	Botão	21
4.2.2	Lógica de Controle	21
4.3	Circuito Subtrador	22
4.3.1	Lógica de passo	22
4.3.2	Subtrai 1	22
4.3.3	Soma 1	23
4.3.4	Comparadores	24
4.3.5	Montagem do Subtrator	24
4.4	Circuito Liberador	25
4.5	Montagem da Máquina de troco	27
5	Conclusão	28
A	Relato semanal	30
A.1	Equipe	30
A.2	Defina o problema	30
A.3	Pontos-chaves	30
A.4	Questões de pesquisa	30
A.5	Planejamento da pesquisa	30

B	Relato semanal	31
B.1	Equipe	31
B.2	Defina o problema	31
B.3	Pontos-chaves	31
B.4	Questões de pesquisa	31
B.5	Planejamento da pesquisa	31
C	Anexos	32

1 Introdução

Como já antes visto, nos circuitos sequenciais as saídas não são simplesmente determinadas pela combinação lógica das entradas, mas por uma combinação de entradas com o estado anterior do circuito, determinada por uma Máquina de Estados Finitos (FSM). Uma FSM é o conjunto de estados que representa todos os estados de um sistema, desse modo, representam o funcionamento de circuitos sequenciais. Isso acontece pois em um circuito digital, os registradores armazenam bits, e bits armazenados significam que o circuito tem memória, o que também é conhecido como estado, que resultam os circuitos sequenciais com estados. Assim, podemos usar esses estados para projetar circuitos que tenham determinados comportamentos ao longo do tempo (Vahid 2008).

Dessa forma, a utilização de uma FSM torna-se essencial no projeto da máquina de troco, pois esse sistema requer a execução de diversas etapas de forma sequencial e controlada. Cada estado da FSM representa uma fase específica do funcionamento, como o carregamento do valor de entrada, a verificação dos cofres disponíveis, a subtração progressiva do valor total e liberação das moedas correspondentes. Assim, a FSM permite que o circuito altere seu comportamento ao longo do tempo conforme o estado atual e as condições de entradas, garantindo que o processo de troco ocorra de maneira ordenada, lógica e sincronizada com o clock do sistema.

O problema proposto consistiu no desenvolvimento e implementação de uma máquina de troco com os seguintes sinais de entradas e saídas e suas respectivas funções representadas na Tabela 1.1.

Sinais	Pinos	Funções
Entrada T	KEY[3]	Botão sincronizado
Entrada V	SW[0:9]	Entrada do troco do cliente
Entrada c	SW[10:15]	Indicação de cofre vazio
Saída i	LEDR[1:6]	Leds que indicam a liberação de cada moeda
Saída L	LEDG[0]	LED que indica processamento do troco
Entrada reset	KEY[0]	Reset no sistema
Entrada clk	Clock27	Pulso de clock

Tabela 1.1: Sinais de Entradas e Saídas da Máquina de Cofre

A máquina de troco libera em moedas um valor determinado colocado em sua entrada. A liberação das moedas é realizada por um sistema cofre que libera uma moeda sempre que em sua entrada ix existir um nível lógico alto e ocorrer um pulso de clock. A entrada com o valor do troco a ser liberado é realizada através de um número binário V e do pulso gerado a partir da saída do circuito de um botão sincronizado (BS)

cuja entrada é T. Adicionalmente, a máquina possui uma saída L, a qual quando está piscando indica que a máquina está processando a informação para liberar o troco e qualquer outra solicitação de troco será ignorada. A máquina de troco possui ainda a capacidade de verificar se algum dos cofres de moedas está vazio ($cx = 0$) e recalcula o troco para liberar moedas apenas dos cofres que não estão vazios. Por fim, a máquina de troco indicará que não consegue trocar o valor da entrada V mantendo a saída L em nível lógico alto até que um novo valor do troco a ser liberado seja carregado na máquina.

2 Fundamentação Teórica

2.1 Metodologia de Projeto RTL

O projeto em nível de transferência entre registradores, ou projeto RTL, consiste em uma ampla variedade de abordagens. No entanto, geralmente um projetista especifica os registradores de um circuito, descreve as possíveis transferências e operações a serem realizadas com os dados de entrada, de saída e dos registradores, e define o controle que especifica quando transferir e operar com dados.

Para iniciar um projeto em RTL, é importante a criação de uma máquina de estados de alto nível cuja as condições para as transições e as ações dos estados são mais do que operações booleanas envolvendo os bits de entrada e de saída. São necessários 4 passos para projetar um circuito sequencial em RTL, conforma a Tabela 2.1.

Ordem	Etapa	Descrição
1	Obtenha uma máquina de estados de alto nível	Descreva o comportamento desejado do sistema na forma de uma máquina de estados de alto nível, consistindo em estados e transições.
2	Crie um bloco operacional	Partindo da máquina de estados de alto nível, crie um bloco operacional capaz de realizar as operações que envolvem dados.
3	Conecte o bloco operacional a um bloco de controle	Conecte o bloco operacional a um bloco de controle. Conecte também as entradas e saídas booleanas que são externas ao bloco de controle.
4	Obtenha a FSM do bloco de controle	Converta a máquina de estados de alto nível na máquina de estados finitos do bloco de controle. Para isso, substitua as operações que envolvem dados por sinais de controle, que são ativados ou lidos pelo bloco de controle.

Tabela 2.1: Metodologia de projeto de RTL

O bloco que define um circuito sequencial conforme o projeto RTL está ilustrado na Figura 2.1.

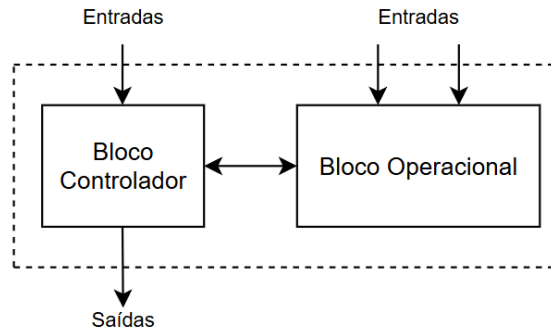


Figura 2.1: Circuito sequencial com RTL

O bloco de controle é essencial em um circuito sequencial projetado em nível de transferência entre registradores. Ele é responsável por coordenar o fluxo de dados dentro do sistema, determinando quando e como os dados devem ser transferidos entre registradores e outras unidades funcionais, como somadores, comparadores, multiplexadores, etc. O bloco de controle é um circuito sequencial que controla saídas booleanas com base em entradas booleanas e em um comportamento específico, ordenado no tempo. Para construir tal bloco, é necessário um registrador de estado, que armazena o estado atual da máquina de controle, e uma lógica combinacional, que determina o próximo estado com base nas entradas atuais e no estado armazenado, e as saídas de controle, que comandam transferências de dados, ativações de módulos ou sinais de alimentação (Vahid 2008).

São necessários 5 passos para projetar um bloco controlador, conforme a Tabela 2.2.

Ordem	Etapa	Descrição
1	Capture a FSM	Crie uma FSM que descreva o comportamento desejado do bloco de controle.
2	Crie a arquitetura	Crie a arquitetura usando um registrador de estado com largura apropriada e uma lógica combinacional, cujas entradas são os bits do registrador de estado e as entradas da FSM e cujas saídas são os bits de próximo estado e as saídas da FSM.
3	Codifique os estados	Atribua um número binário único a cada um dos estados.
4	Crie a tabela de estados	Crie uma tabela-verdade para a lógica combinacional de modo tal que a lógica irá gerar as saídas e os sinais de próximo estado corretos para a FSM.
5	Implemente a lógica combinacional	Implemente a lógica combinacional usando qualquer método.

Tabela 2.2: Metodologia de projeto de bloco de controle

2.2 Demultiplexadores

Demultiplexadores são componentes que recebem uma única entrada de dados e a distribuem para uma de várias saídas, de acordo com os sinais de seleção. Também podem ser chamados de distribuidores, pois direcionam o valor de entrada para a saída selecionada. Este controle é feito por meio de entradas de seleção.

Um demultiplexador de 1 entrada e 2 saídas, conhecido como DEMUX 1x2, possui uma entrada de dados d , uma entrada de seleção s_0 e duas saídas y_0 e y_1 . Quando $s_0 = 0$, o valor de d é direcionado para y_0 e y_1 permanece em 0. Quando $s_0 = 1$, o valor de d é direcionado para y_1 e y_0 permanece em 0. A tabela-verdade que representa este componente está ilustrada abaixo.

Tabela 2.3: Tabela-verdade de um DEMUX 1x2

s_0	d	y_0	y_1
0	1	1	0
0	0	0	0
1	1	0	1
1	0	0	0

As expressões lógicas das saídas y_0 e y_1 são dadas por:

$$y_0 = d \cdot s_0' \quad \text{e} \quad y_1 = d \cdot s_0$$

Para demultiplexadores de ordem superiores é possível fazer o empilhamento de demultiplexador de ordem baixa, conforme a Fig.2.2

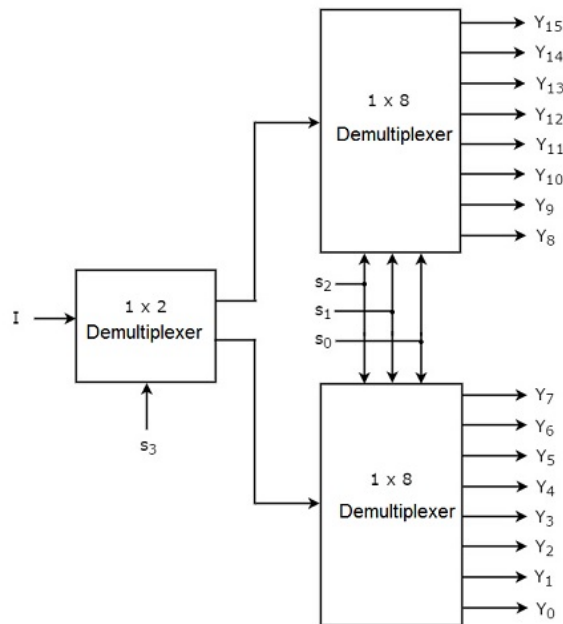


Figura 2.2: Demux 1x16 Fonte: (TP 2025)

3 Desenvolvimento da Solução

3.1 Botão Síncrono

O botão síncrono T foi projetado de modo que quando o botão for apertado, o resultado será um sinal que fica em nível alto por um ciclo de relógio. Ele é útil para evitar que um simples aperto de botão, que pode durar diversos ciclos de relógio, seja interpretado como múltiplos apertos. Este sistema tem entrada bi , representando o aperto no botão, e saída $b0$. A FSM inicial foi projetada e, posteriormente, os estados $S0$, $S1$ e $S2$ foram respectivamente codificados pelos números binários 00, 01 e 10, conforme ilustrado na Figura 3.1.

O funcionamento desta máquina de estados acontece por meio das seguintes etapas:

- 1) A FSM fica esperando no estado $S0$ gerando uma saída $b0=0$ até que bi seja 1;
- 2) Quando $bi=1$, a FSM faz a transição para o estado $S1$, produzindo a saída $b0=1$.
- 3) A seguir, a FSM faz uma transição para o estado $S0$ ou $S2$, os quais ambos geram $b0=0$ novamente. Desse modo, $b0$ foi 1 por apenas um ciclo. Se bi retornar a 0, a FSM passará de $S1$ para $S0$. Se bi ainda for 1, a FSM irá para o estado $S2$, no qual fica esperando que bi retorne a 0, causando uma transição de volta ao estado $S0$.

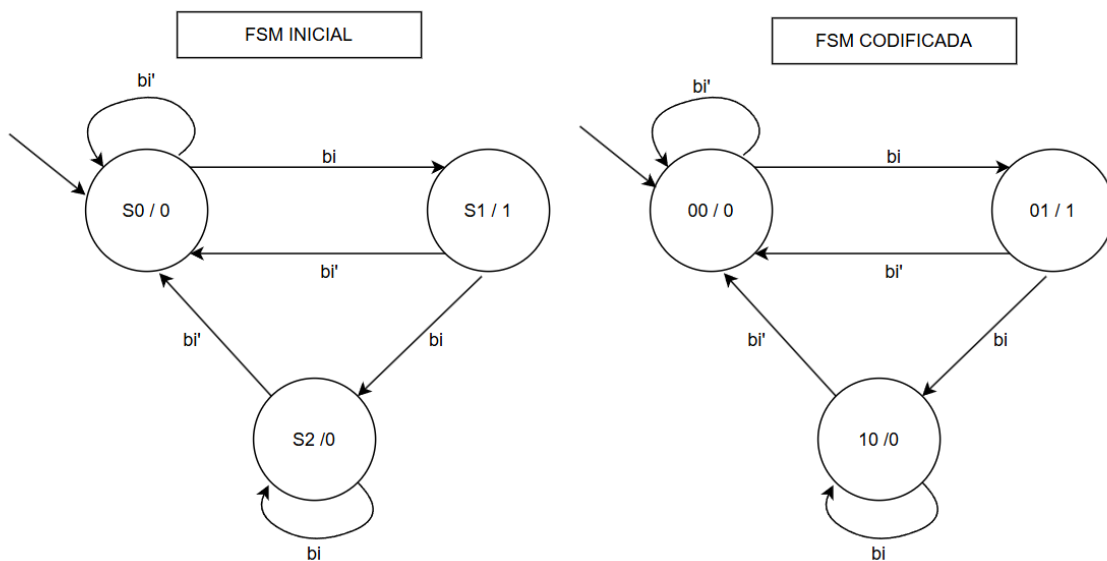
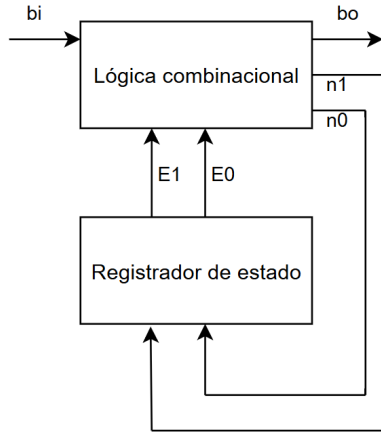


Figura 3.1: FSM inicial e codificada do botão síncrono

Desse modo, concluímos que a FSM tem 3 estados que podem ser representados por 2 bits. Os estados atuais são denominados $E1$ e $E0$ e os estados futuros são denominados

$n1$ e $n0$. Assim, são necessários 2 registradores para armazenar estes estados.

A arquitetura desse sistema é descrita de acordo com a Figura 3.2 e a tabela-verdade que contém a lógica combinacional dos estados é descrita na Tabela 3.1.



E1	E0	bi	n1	n0	b0
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	0	1
1	0	0	0	0	0
1	0	1	1	0	0
1	1	x	x	x	x

Tabela 3.1: Tabela-verdade do botão síncrono

Figura 3.2: Arquitetura do Botão Síncrono

No qual, as expressões lógicas por cada saída são:

$$b0 = E0$$

$$n0 = E1' \cdot E0' \cdot bi$$

$$n1 = E1 \cdot bi + E0 \cdot bi$$

O circuito lógico está ilustrado na Figura 3.3.

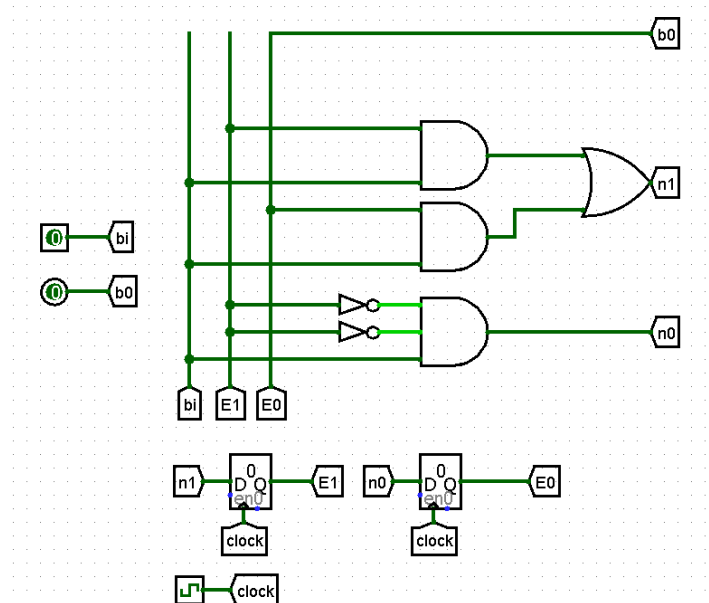


Figura 3.3: Circuito lógico do Botão síncrono

3.2 Máquina de troco

A máquina de Troco foi inicialmente desenvolvida conforme a Máquina de Estados de Alto Nível, de apenas 2 bits e 4 estados, visando simplificar a abstração do problema. Entretanto, esta simplificação do número de estados gerou uma maior complexidade no bloco operacional, conforme será discutido posteriormente. A FSM está apresentada na Figura 3.4.

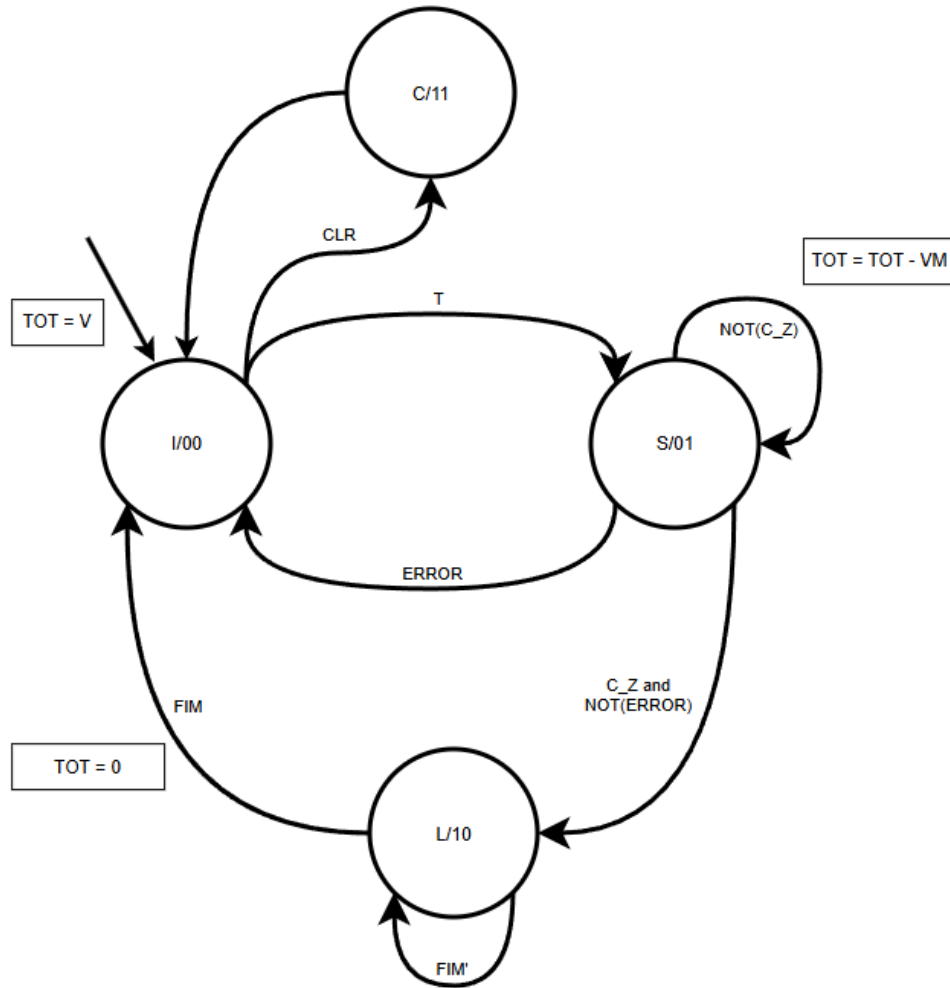


Figura 3.4: Máquina de Estado de Alto Nível

O sistema inicia no estado I (Estado de Início), no qual o valor binário de 10 bits, correspondente a valores entre 0 e 1000 (R\$0,00 a R\$10,00), é carregado no registrador TOT. Esse registrador é responsável por armazenar o valor que será posteriormente operado pelo subtrator. Nesse estado, a máquina pode transitar para dois outros estados: o estado C (Estado de Clear) ou o estado S (Estado de Subtração).

O estado C é responsável por redefinir os valores iniciais dos registradores correspondentes aos cofres de moedas, garantindo a inicialização adequada do sistema.

Por sua vez, o estado S é encarregado de subtrair valores do total armazenado, conforme a disponibilidade de moedas no cofre. A cada operação de subtração, segue a seguinte lógica:

- Subtrair do total o valor da moeda de maior valor possível;
- Subtrair uma unidade do cofre de moedas correspondente;
- Somar uma unidade no cofre de liberação;
- Comparar o valor restante da subtração com zero, gerando o sinal **C_Z**;
- Gerar o sinal **ERROR** caso o valor a ser subtraído seja maior que zero e não haja moedas suficientes no cofre correspondente.

Caso o sistema esteja no estado S e receba o sinal **C_Z** em nível lógico HIGH, independente do **ERROR**, ele irá para o estado L (Estado de Liberação). Por outro lado, caso receba o sinal **ERROR** em nível lógico HIGH, o sistema retornará ao estado I, indicando que não há troco suficiente disponível nos cofres.

O estado L (Estado de Liberação) é responsável pela liberação das moedas. A cada ciclo de operação, uma moeda é liberada de cada cofre que o valor seja superior a zero. Simultaneamente, a cada liberação, é subtraída uma unidade do cofre de liberação correspondente, garantindo a atualização correta do saldo de moedas disponíveis.

Quando todos os cofres estão vazios o circuito gera um sinal **FIM HIGH**, e o circuito volta para o estado I.

3.2.1 Bloco Operacional

O bloco operacional está ilustrado na Figura 3.5. Ele possui duas entradas principais: **S_1**, um sinal de 2 bits que indica o estado atual da máquina, e o valor V, correspondente à entrada de 10 bits. Ele apresenta três saídas: os sinais **ERROR**, **C_Z** e o sinal **FIM** proveniente do bloco Liberador.

Dentro do bloco operacional há três conjuntos de registradores, descritos a seguir:

- Registrador TOT: responsável por armazenar o valor total a ser operado pelo sistema;
- Registradores do cofre de moedas: conjunto de seis registradores de 3 bits, onde são armazenados os valores atuais de cada cofre de moedas;
- Registradores do cofre de liberação: conjunto de seis registradores de 3 bits, responsáveis por armazenar os valores referentes aos cofres de liberação.

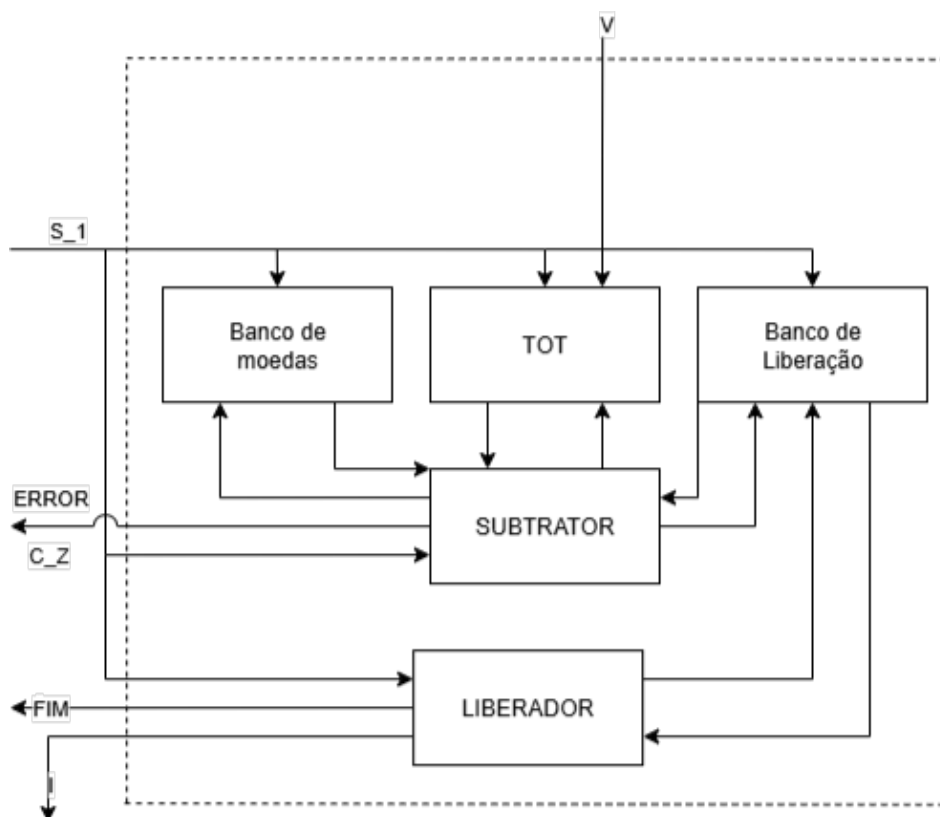


Figura 3.5: Bloco Operacional

Além desses componentes, o bloco operacional também é composto pelos módulos SUBTRATOR e LIBERADOR, responsáveis, respectivamente, pelas operações de subtração e de liberação de moedas.

Subtração

No bloco de SUBTRAÇÃO, as entradas consistem nos valores dos cofres de moedas, cofres de liberação, o valor total (TOT) e o sinal **ENABLE**. O circuito realiza a comparação de cada cofre de moedas individualmente com zero, verificando também se o valor de cada moeda é menor ou igual ao valor total armazenado em TOT. Caso o valor total seja maior ou igual ao valor da moeda correspondente, o sinal associado àquela moeda é definido como HIGH, gerando assim os sinais **C1** a **C6**.

Com os sinais **C1** a **C6** e o **ENABLE** é possível decidir o passo da subtração, se há **ERROR**, e verificar o cofre de moedas que deverá ser subtraído uma unidade e o cofre de liberação que deverá ser somado uma unidade. Essa lógica está descrita na Tabela 3.2.

O passo é dado em binário, sendo o maior valor 100 em decimal (1100100 em binário). Com essa lógica a moeda selecionada para subtração sempre será a maior possível para que não gere valor negativo na subtração. As saídas M1-M6 e C correspondem ao cofre das moedas que será subtraído/somado. Caso o valor seja LOW o cofre das moedas será subtraído/somado, caso seja HIGH o valor é mantido.

Tabela 3.2: Tabela de Verdade Passo

Entradas							Saídas								
C1	C2	C3	C4	C5	C6	ENABLE	C[2:0]	PASSO	ERROR	M1	M2	M3	M4	M5	M6
-	-	-	-	-	-	0	111	0000000	0	1	1	1	1	1	1
0	0	0	0	0	0	1	111	0000000	1	1	1	1	1	1	1
0	0	0	0	0	1	1	101	0000001	0	1	1	1	1	1	0
0	0	0	0	1	-	1	100	0000101	0	1	1	1	1	0	1
0	0	0	1	-	-	1	011	0001010	0	1	1	1	0	1	1
0	0	1	-	-	-	1	010	0011001	0	1	1	0	1	1	1
0	1	-	-	-	-	1	001	0110010	0	1	0	1	1	1	1
1	-	-	-	-	-	1	000	1100100	0	0	1	1	1	1	1

O sinal de **ERROR** só ocorre quando não há mais troco, ou seja ou todos os cofres de moedas estão vazias, ou as moedas atuais não conseguem realizar o troco. O sinal de **C_Z** é HIGH quando a subtração é negativo ou igual a zero.

Como o sistema não opera em sinais negativos é necessário corrigir nesses casos, senão é possível que moedas sejam erroneamente operadas. Para corrigir este caso foi usado um MUX que é controlado pelo carry_out do subtrator, caso esse sinal seja HIGH é gravado o valor 0 na saída do TOT, caso LOW é gravado o valor de TOT - PASSO.

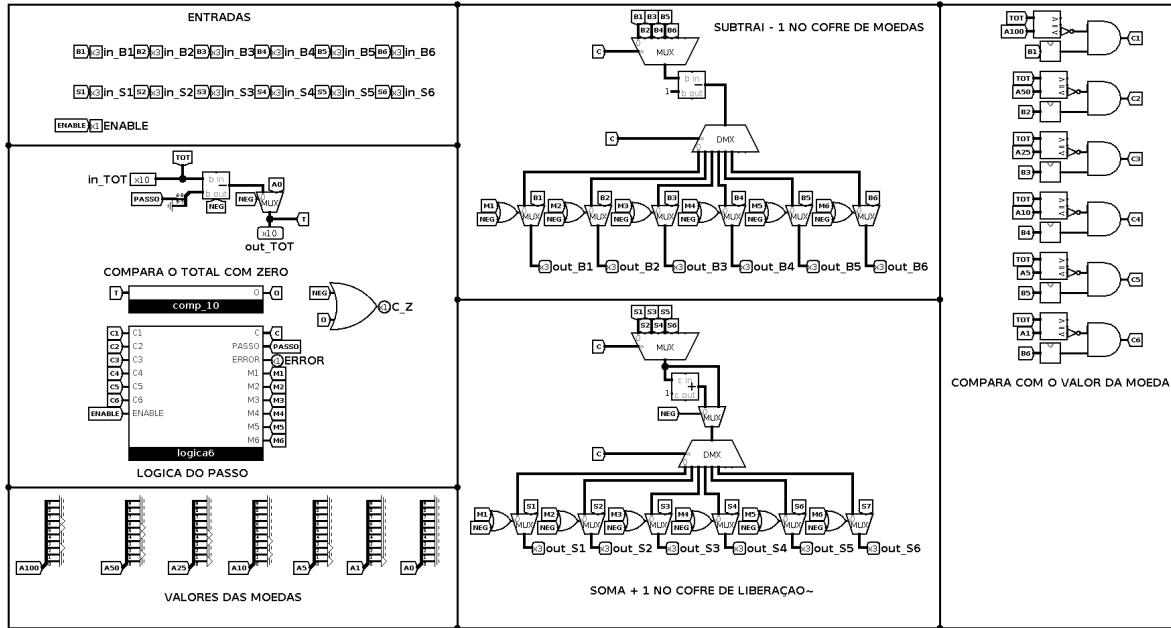


Figura 3.6: Circuito Subtrator

Nos circuitos de subtrações e adições dos cofres de moedas e liberação respectivamente é realizado por um arranjo de um MUX e um DEMUX, com um subtrator e um somador de 1 unidade no meio. Após esse arranjo há um MUX extra que define se o valor é mantido ou subtraído conforme a Figura 3.6.

Liberação

O bloco de liberação tem como objetivo realizar a liberação das moedas acumuladas nos cofres de liberação. Para isso, a cada operação, cada cofre é individualmente

comparado com zero por um AND, caso seja diferente de zero o seu valor é reduzido uma unidade por um subtrator e um sinal **I1** a **I6** correspondente ao cofre é posto no estado HIGH, caso seja igual a zero o valor é mantido e o sinal **I1-I6** é posto em LOW, conforme Figura 3.7.

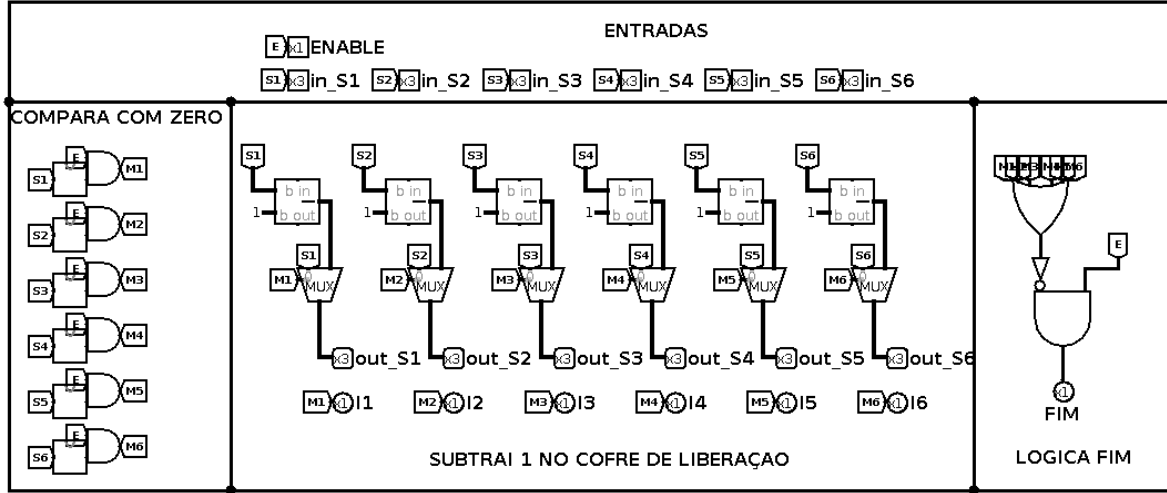


Figura 3.7: Circuito Liberador

O sinal **FIM** só sera posto em HIGH caso todos os cofres de liberação estejam vazios e o **ENABLE** esteja em HIGH.

3.2.2 Bloco de Controle

O bloco de controle é responsável por mudar os estados do bloco de operacional, ou seja no estado de $C_Z = 00$, o bloco operacional apenas grava o valor de entrada V no registrador TOT . Quando no estado 01 irá operar usando o bloco **SUBTRATOR**, quando no estado 10 irá operar usando o bloco **LIBERADOR**, e quando no estado 11 irá realizar o clear dos cofres de moedas e a liberação delas.

O bloco de controle opera a cada clock, e sempre é atualizado o valor do estado atual nos registradores de estado, nesse caso de 2 bits. Para decidir qual estado é o próximo (S_1), é feita uma relação entre o estado do clock anterior (S_0) e as entradas S_0 , $ERROR$, C_Z , FIM , T e CLR .

A tabela-verdade do bloco de controle segue a Tabela 3.3.

Tabela 3.3: Tabela da Verdade dos Sinais de Controle

ERROR	C_Z	FIM	T	S ₀ [1..0]	CLR	S ₁ [1..0]
-	-	-	0	0	0	0 0
-	-	-	0	0	1	1 1
0	0	-	-	0	-	0 1
-	-	0	-	1	-	1 0
-	-	-	0	1	0	0 0
-	-	-	0	1	1	1 1
-	-	-	1	0	-	0 1
-	-	-	1	1	-	0 1
-	-	1	-	1	-	0 0
-	1	-	-	0	1	1 0
1	0	-	-	0	1	0 0

Seguindo a lógica, caso a máquina esteja no estado 00, ela só irá para o estado 01 se o sinal T, do botão, estiver em HIGH, independentemente dos outros sinais. Irá para o estado 11 caso o sinal CLR esteja em HIGH e T em LOW.

No estado 01, a máquina só irá transitar para o estado 00 caso o sinal ERROR esteja em HIGH e C_Z em LOW, indicando que não há troco suficiente. Irá para o estado 10 caso C_Z esteja em HIGH, independentemente dos outros sinais.

No estado 10, a máquina só pode ir para 00; isso ocorrerá quando o sinal FIM estiver em HIGH, independentemente dos outros sinais.

No estado 11, a máquina só poderá transitar para o estado 00, e isso ocorrerá quando C_Z estiver em LOW.

Resolvendo a tabela verdade 3.3 temos as seguintes expressões:

- $S_1(1) = [C_Z + S_0(1) + \overline{S_0(0)}] * [\overline{FIM} + S_0(1) + S_0(0)] * [S_0(1) + S_0(0) + \overline{CLR}] * [\overline{S_0(1)} + S_0(0) + CLR] * [\overline{T} + S_0(1) + S_0(0)] * [T + S_0(1) + S_0(0)]$
- $S_1(0) = [T + S_0(0) + \overline{CLR}] * [\overline{S_0(1)} + S_0(0)] * [T + \overline{S_0(1)} + \overline{CLR}] * [\overline{C_Z} + S_0(1) + S_0(0)] * [\overline{ERROR} + S_0(1) + S_0(0)]$

O circuito implementado segue conforme a Figura 3.8.

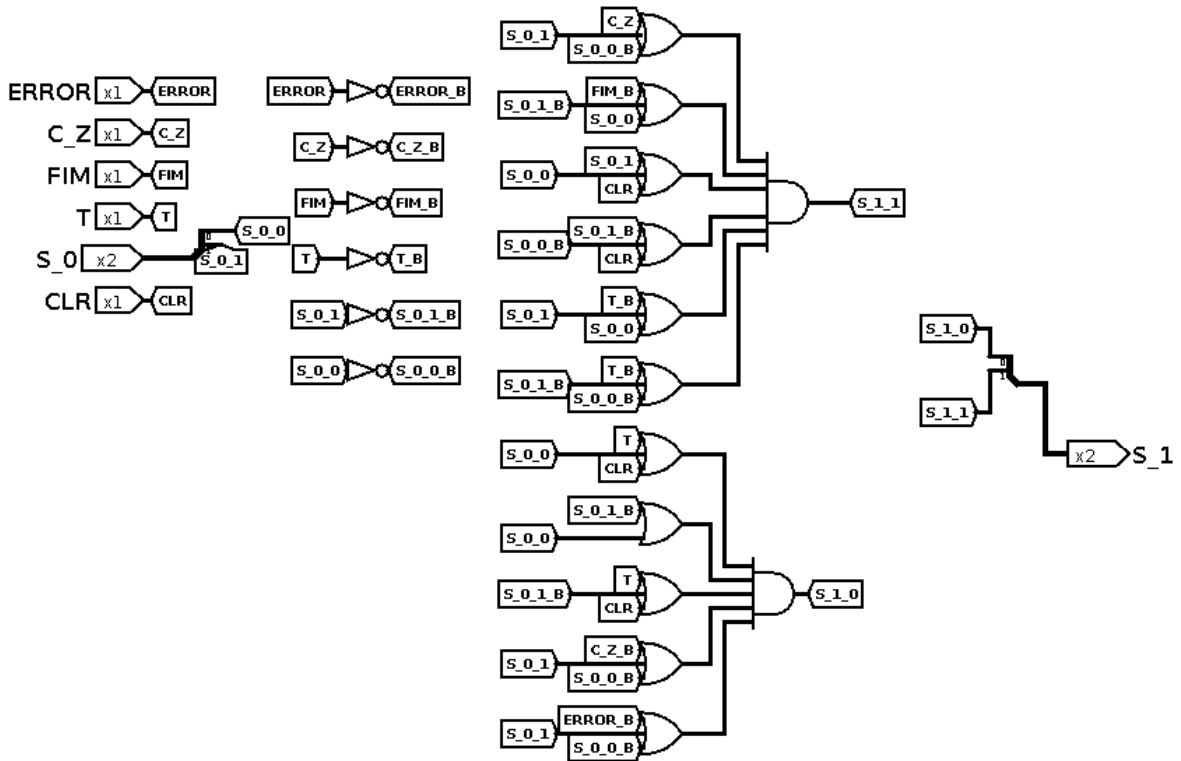


Figura 3.8: Circuito da lógica de controle

Como é necessário armazenar o valor do estado atual, para o próximo ciclo foi utilizado um registrador de 2 bits. Desse modo, a montagem do circuito segue a Figura 3.9.

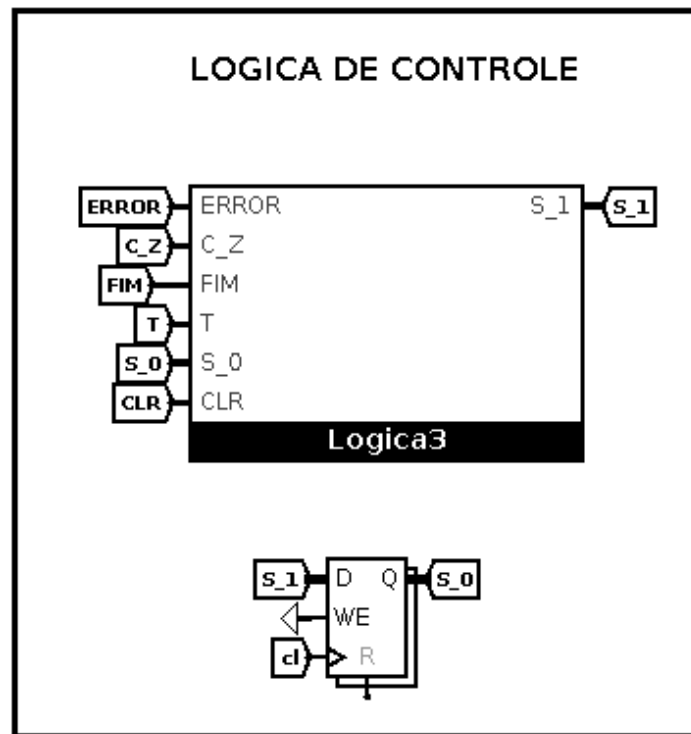


Figura 3.9: Bloco de Controle

3.3 Ligação do Operacional com o Controle

Conforme discutido anteriormente é necessário conectar o bloco de operação com o bloco de controle, seguindo a lógica da Figura 3.5 é possível conectar os dois blocos, a Figura 3.11 mostra a montagem do circuito, onde o bloco de controle e bloco de operação são conectados.

Os registradores de moedas e de liberação são ambos de 3 bits com valor máximo de 7 em decimal, assim é possível que as moedas do cofre de moedas se esgotem, podendo testar a logica da maquina de troco completamente. O registrador TOT é de 10 bits. Os valores gravados nos registradores depende do valor do estado atual S_0, seguindo a seguinte logica:

Cofre de moedas (6 registradores de 3 bits)

- S_0 (00) Não altera;
- S_0 (01) Grava o valor que sai do bloco de subtração;
- S_0 (10) Não altera;
- S_0 (11) Grava o valor de clear (7);

Cofre de liberação (6 registradores de 3 bits)

- S_0 (00) Não altera;

- S_0 (01) Grava o valor que sai do bloco de subtração;
- S_0 (10) Grava o valor que sai do bloco de liberação;
- S_O (11) Grava o valor de clear (0);

Registrador de TOT (1 registrador de 10 bits)

- S_0 (00) Grava o valor de V (valor de entrada);
- S_0 (01) Grava o valor que sai do bloco de subtração;
- S_0 (10) Não altera;
- S_O (11) Não altera;

A conexão entre os blocos operacionais e de controle estão ilustrados na Figura 3.10.

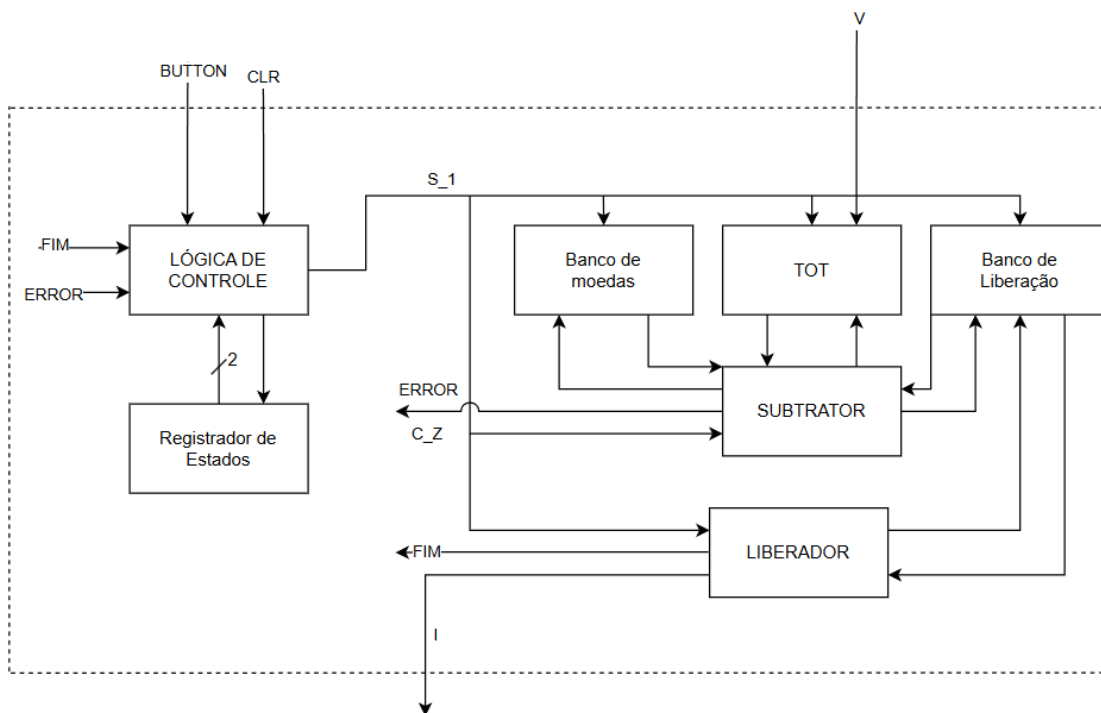


Figura 3.10: Bloco Operacional e de Controle

Além disso, como é requisito de projeto gerar um sinal HIGH de **C1** a **C6** quando os cofres estão vazios foi adicionado um comparador com zero em todos os cofres de moedas. Assim a montagem fica a seguinte:

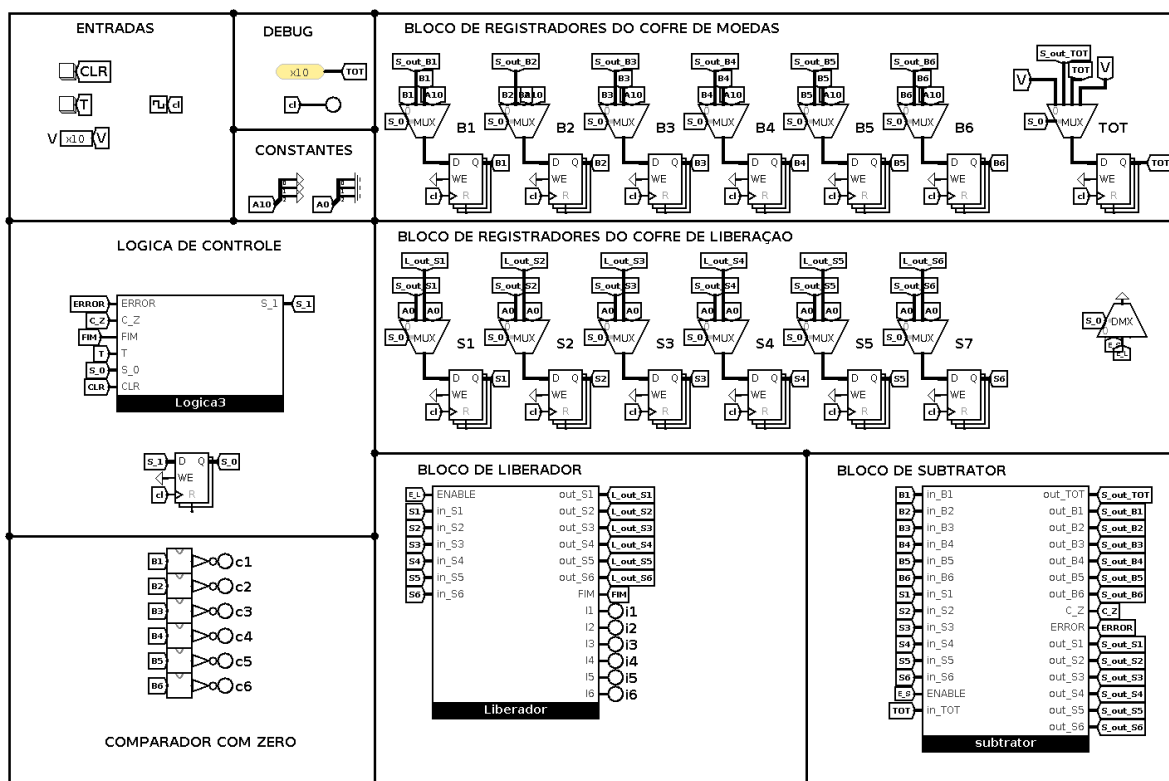


Figura 3.11: Máquina de troco

4 Implementação

4.1 Correção de projeto

Como no projeto não foi especificado como o sinal L, que indica que a máquina está a processar o troco, foi definido que o bloco de controle irá gerar esse sinal, quando estiver no estado "01" e "10" que corresponde aos estados de subtração e liberação respectivamente. Além disso agora o sinal de error é guardado em um registrador, e esse persiste no estado HIGH até ser restaurado no estado de clear, esse comportamento também é gerenciado pelo bloco de controle.

4.2 Máquina de Estados

4.2.1 Botão

A máquina de estados do botão síncrono foi implementada em VHDL a partir do modelo fornecido pelo professor, adaptado à lógica do projeto. O circuito utiliza três estados (A, B e C) correspondentes aos estados S0, S1 e S2 descritos anteriormente. No estado A, o sistema permanece inativo aguardando o acionamento do botão. Ao detectar $s = '1'$, ocorre a transição para B, gerando um pulso de apenas um ciclo de clock. Caso o botão permaneça pressionado, o sistema passa para C, retornando a A quando o botão é liberado.

A implementação utiliza um processo sensível ao clock e ao sinal de reset, garantindo sincronização e inicialização previsível no estado A. Os estados são codificados em um vetor de dois bits (q), enquanto a saída b0 representa o pulso gerado durante o estado B. Assim, o circuito assegura que cada acionamento físico produza apenas um pulso lógico. Esse módulo é fundamental para o funcionamento da máquina de troco, uma vez que seu sinal de saída é utilizado como entrada de controle para o início do processamento do valor de troco e fornece o sinal de início de operação de forma sincronizada, evitando múltiplos disparos indevidos e garantindo o controle confiável do sistema.

4.2.2 Lógica de Controle

O bloco de controle foi implementado em VHDL com base na tabela-verdade e na máquina de estados descritas na Seção 3.2.2. Esse módulo é responsável por definir a sequência de operações do sistema, determinando quando o bloco operacional deve realizar a gravação de valores, a subtração, a liberação de moedas ou a limpeza dos registradores.

A FSM é composta por quatro estados: I (início), S (subtração), L (liberação) e C (clear). O estado I corresponde à condição inicial, aguardando o sinal de ativação do botão (**T**) para iniciar o processo de cálculo do troco. No estado S, são realizadas as operações de subtração do valor total. Quando a subtração é concluída (**CZ = '1'**), a FSM avança para o estado L, responsável pela liberação das moedas. Por fim, o estado C executa a limpeza dos cofres quando o sinal de controle (**CRL**) é ativado.

O processo principal é sensível ao clock e ao sinal de reset, garantindo que a FSM retorne ao estado I sempre que o sistema for reinicializado. A codificação dos estados é representada no vetor de saída q (1 downto 0), utilizado para controlar o comportamento do bloco operacional de acordo com o estado atual. Assim, o código VHDL implementa de forma fiel a lógica sequencial definida no projeto, assegurando o funcionamento sincronizado entre os módulos da máquina de troco.

4.3 Circuito Subtrador

O circuito subtrator tem como objetivo controlar as operações que ocorrem no estado de subtração, onde o valor total é subtraído de acordo com a disponibilidade de moedas no cofre, sendo subtraída uma moeda do cofre de moedas correspondente e adicionada uma moeda ao cofre de liberação. Esse circuito foi implementado em vários blocos para facilitar a implementação em VHDL, sendo dividido em 4 partes: a lógica de passo, comparadores, o circuito "subtrai 1"(circuito que deduz uma moeda do cofre de moedas) e o circuito "soma 1"(circuito que soma uma moeda ao cofre de liberação).

4.3.1 Lógica de passo

O circuito logica_passo foi implementado em VHDL para controlar a sequência de operações do sistema de troco. A lógica considera seis entradas C1 a C6, que representam condições do sistema, e um sinal ENABLE, que ativa a operação do módulo. O segue a tabela verdade descrita em 3.2. A simulação pode ser observada na Fig.4.1

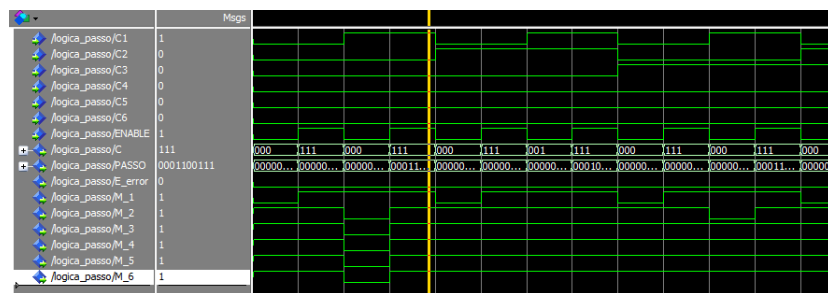


Figura 4.1: Lógica passo

4.3.2 Subtrai 1

O módulo sub_1_cofre realiza a subtração de uma unidade em cada cofre de moedas selecionado, de acordo com o sinal de controle do sistema. Ele recebe como entradas os valores atuais dos cofres (B1 a B6), sinais de habilitação (M1 a M6), o sinal de negação

(NEG) e o seletor de cofre (C). As saídas (B_out1 a B_out6) representam os novos valores dos cofres após a operação.

O funcionamento do módulo ocorre em três etapas principais. Primeiro, os sinais de habilitação são combinados com o sinal NEG através de um OR lógico, garantindo que a atualização dos cofres ocorra conforme a condição de habilitação. Em seguida, um multiplexador 8x3 seleciona o valor do cofre correspondente ao seletor C, que é então subtraído de 1 por um somador de 3 bits configurado como decrementador. O resultado é redistribuído pelo demultiplexador 3x8 para o cofre correspondente e, por fim, cada cofre é atualizado por um multiplexador 2x3 que escolhe entre o valor subtraído ou o valor original, de acordo com o sinal de habilitação. Conforme descrito no projeto.

A simulação pode ser observada na Fig.4.2

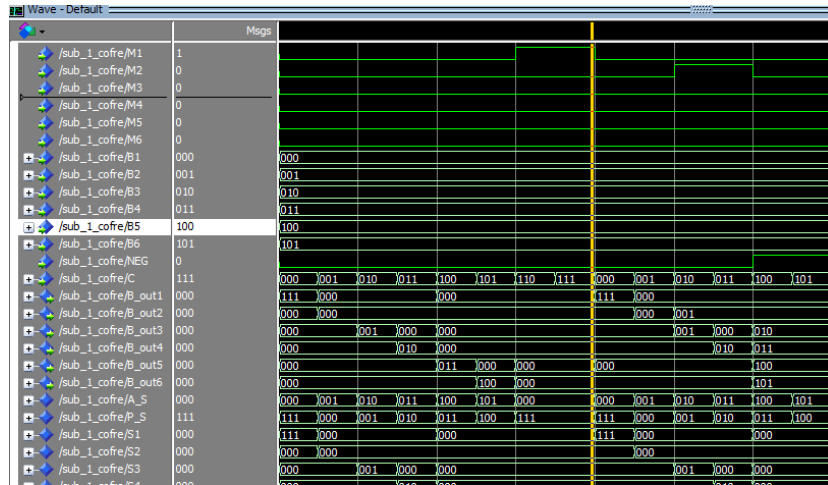


Figura 4.2: subtrai 1

4.3.3 Soma 1

O módulo soma_1_cofre realiza a adição de uma unidade em um cofre de liberação selecionado, de acordo com os sinais de habilitação (M1 a M6), o seletor de cofre (C) e o sinal de negação (NEG). Ele recebe como entradas os valores atuais dos cofres de liberação (S1 a S6) e gera como saída os novos valores (S_out1 a S_out6) após a operação.

O funcionamento é semelhante ao módulo sub_1_cofre. Primeiro, cada sinal de habilitação é combinado com NEG através de um OR. Um multiplexador 8x3 seleciona o valor do cofre correspondente ao seletor C, que é incrementado em 1 por um somador de 3 bits. O resultado é redistribuído pelo demultiplexador 3x8 e cada cofre é atualizado por multiplexadores 2x3, escolhendo entre o valor incrementado ou o valor original conforme o sinal de habilitação. A simulação pode ser observada na Fig.4.3

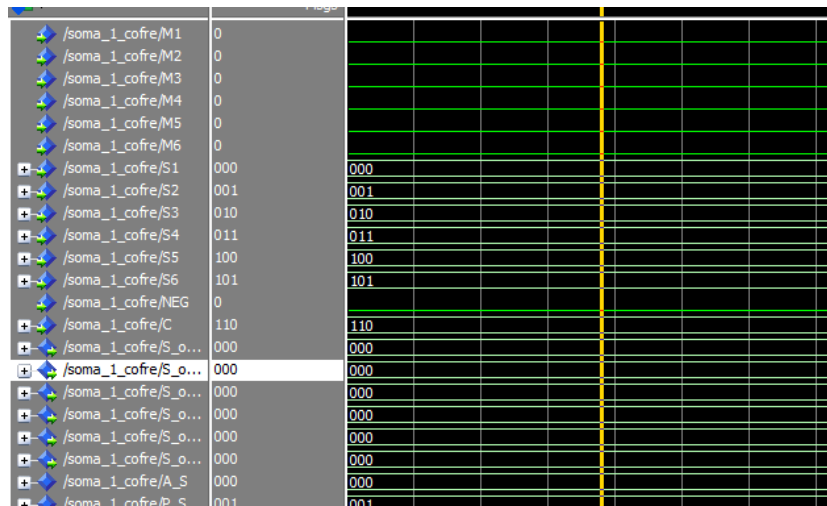


Figura 4.3: soma 1

4.3.4 Comparadores

O módulo comp_moeda tem a função de verificar se a quantidade total inserida (TOT) é suficiente para cada tipo de moeda disponível nos cofres. Ele recebe como entradas o valor total (TOT) e os saldos atuais dos cofres (B1 a B6), e gera sinais de controle (C1 a C6) indicando se cada cofre pode ser utilizado na liberação de moedas.

A lógica do módulo utiliza dois tipos de comparadores. Primeiramente, comparadores de 10 bits verificam se o valor total é menor que os limites correspondentes a cada moeda, produzindo sinais intermediários (S1 a S6). Em seguida, comparadores de magnitude de 3 bits testam se cada cofre possui moedas disponíveis, gerando sinais M1 a M6. Finalmente, cada saída de controle C1 a C6 é obtida pela conjunção lógica entre a disponibilidade do cofre e a condição do valor total, assegurando que apenas cofres com saldo e com valor suficiente sejam considerados para liberação. A simulação pode ser observada na Fig.4.4

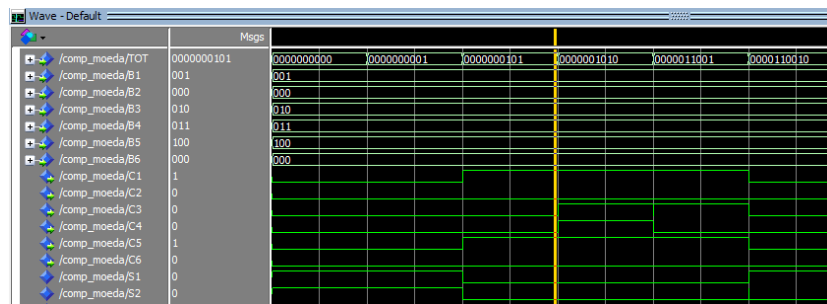


Figura 4.4: Comparadores

4.3.5 Montagem do Subtrator

O módulo subtrator realiza a operação completa de subtração e atualização dos cofres de moedas. Ele é composto pela interligação dos módulos citados anteriormente. Os sinais de controle e os valores intermediários são compartilhados entre os blocos,

garantindo a operação desejada do sistema de troco, de forma que os cofres sejam atualizados conforme foi definido no projeto.

4.4 Circuito Liberador

O circuito liberador tem função de controlar a liberação das moedas armazenadas nos cofres de liberação, reduzindo a contagem dos cofres e ativando os sinais de saída de liberação. Ele foi implementado de forma modular, utilizando comparadores de zero, subtratores e multiplexadores 2x1. Para isto, a implementação foi dividida nos componentes *Comparar*, cuja função é determinar se cada cofre de liberação contém moedas a serem dispensadas, e *Subtrair*, cuja função é executar a atualização do saldo dos cofres de liberação.

Inicialmente, o bloco de comparação representada pelo componente *Comparar* foi implementado utilizando os sinais de entrada **Sinx**, que representam o cofre com a quantidade de moedas armazenadas nos cofres de liberação, e realiza a comparação individual destas quantidades com 0. Para tal, a cada operação, cada cofre é individualmente comparado com zero por um AND. Caso o valor armazenado nos cofres seja diferente de 0, o sinal de saída **Msig(x)** assume valor lógico 1, representando que o cofre tem moedas. A arquitetura deste bloco é representada na Figura 4.5.

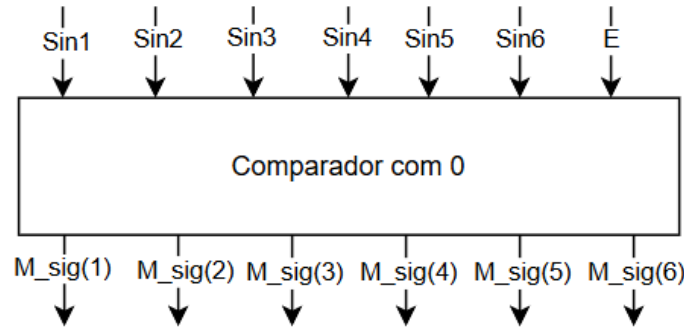


Figura 4.5: Bloco de Comparação

O resultado desta comparação, representado pelo sinal **Msig(x)** é utilizado na entidade *Subtrair* como seletor para um multiplexador. Se **Msig(x) = 1** (cofre tem moedas para serem liberadas), é reduzido em uma unidade de cada cofre por um subtrator e este novo valor é direcionado para a saída **Sout(x)**, garantindo a atualização correta do saldo de moedas disponíveis. Simultaneamente, o sinal de liberação correspondente *I1* a *I6* é colocado no estado HIGH se a operação estiver habilitada com **E = 1**. Caso **Msig(x) = 0**, o cofre está vazio e o valor atual do cofre é mantido e o sinal de liberação correspondente *I1* a *I6* é colocado em LOW. Esta arquitetura é ilustrada nas Figuras 4.6 e 4.7.

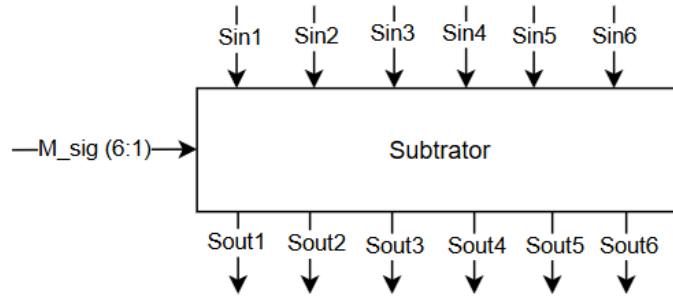
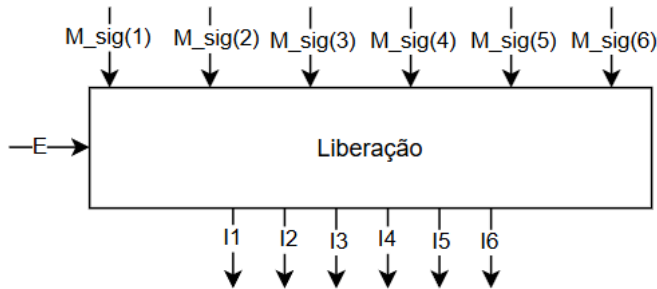


Figura 4.6: Bloco de Subtração



E	Mx	Ix
0	x	0
1	1	0
1	1	1

Tabela 4.1: Lógica de liberação

Figura 4.7: Bloco de Liberação

O sinal de **FIM** é ativado para sinalizar ao bloco de controle que o processo de liberação do troco terminou. Ele é posto em HIGH somente quando o **ENABLE** está em HIGH e todos os cofres de liberação estiverem vazios. O resultado da saída está ilustrada na Figura 4.8.

/liberador/E	1	100	000				
/liberador/Sin1	100	001	000				
/liberador/Sin2	001	010	000				
/liberador/Sin3	010	000	111	000			
/liberador/Sin4	000	011	011	000			
/liberador/Sin5	111	011	000	110	000		
/liberador/Sin6	011	010	010	000			
/liberador/Sout1	011	111011	000000				
/liberador/Sout2	000	0					
/liberador/Sout3	001	111011	000000				
/liberador/Sout4	000						
/liberador/Sout5	110						
/liberador/Sout6	010						
/liberador/I1	111011						
/liberador/FIM	0						
/liberador/M_sig	111011						

Figura 4.8: Saída do Circuito de Liberação

4.5 Montagem da Maquina de troco

A maquina de troco foi montada utilizando os blocos discutidos anteriormente é também registradores, que por ter sido abordado sua implementação em relatórios passados não foi detalhadamente explicado. No entanto a montagem segue a logica discutida no capitulo de projeto, onde os diversos blocos são juntos com sinais internos. A simulação da maquina de troco está disponível na Fig. 4.9.

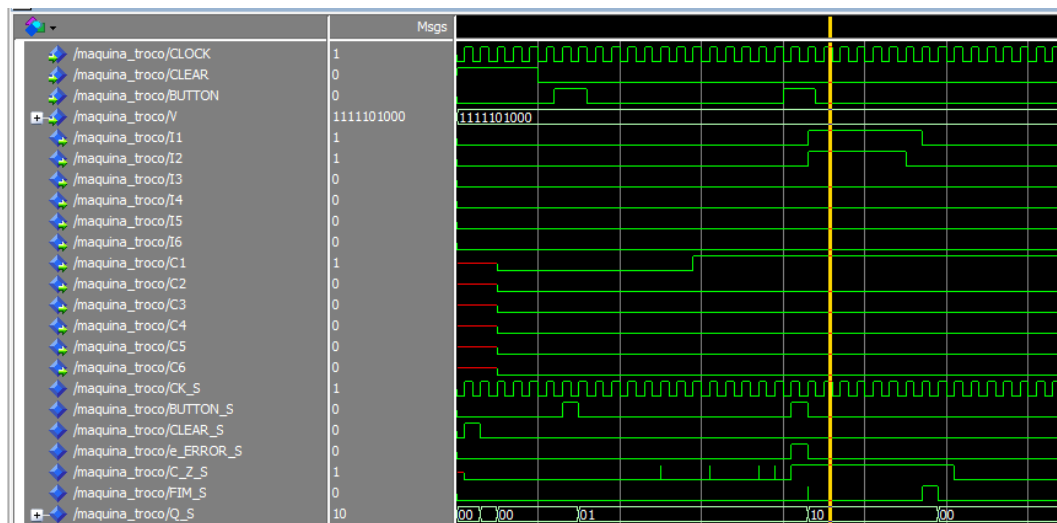


Figura 4.9: Simulação Maquina de troco

No final foi criado uma outra entidade, chamada de !main!, para inverter a logica conforme necessário, pois o kit cyclone 2 tem como nivel logico invertido da logica projetada.

5 Conclusão

O desenvolvimento da máquina de troco permitiu consolidar os conhecimentos teóricos da disciplina de Sistemas Digitais por meio de uma aplicação prática voltada ao projeto RTL. O sistema foi estruturado em blocos, incluindo registradores para armazenar valores de moedas, total acumulado e troco a ser fornecido, além da unidade de controle principal, o que facilitou a validação de cada bloco e a integração em um sistema completo.

Foram utilizados elementos combinacionais, como somadores e multiplexadores, e sequenciais, como Flip-Flops do tipo D. Além disso, foi aplicado o projeto RTL desde uma máquina de estado de alto nível até a simulação no software Logisim.

Os resultados obtidos confirmaram que a máquina de troco atendeu plenamente aos requisitos estabelecidos, sendo capaz de receber diferentes valores de moedas, calcular automaticamente o troco e liberar a quantidade correta.

Referências Bibliográficas

TP 2025 TP. *Digital Electronics - Demultiplexers*. 2025. Acesso em: 13 out. 2025. Disponível em: <<https://www.tutorialspoint.com/digital-electronics/digital-electronics-demultiplexers.htm>>.

Vahid 2008 VAHID, F. *Sistemas digitais: projeto, otimização e HDLs*. Porto Alegre: Artmed, 2008. 560p; 25 cm. ISBN 978-85-7780-190-9.

A Relato semanal

A.1 Equipe

Função no grupo	Nome completo do aluno	Responsável por
Líder	Nicolas Pinheiro Silva	Função: Líder.
Redator	Karen Helloisa Araújo Silva	Funções: Relatório.
Debatedor	Rinaldo Tavares da Silva Filho	Funções: Relatório.

A.2 Defina o problema

Projeto de um maquina de troco.

A.3 Pontos-chaves

Projeto RTL, maquina de estados de alto nível;

A.4 Questões de pesquisa

Projetos RTL, maquina de estados de alto nível;

A.5 Planejamento da pesquisa

([Vahid 2008](#)) apenas.

B Relato semanal

B.1 Equipe

Função no grupo	Nome completo do aluno	Responsável por
Líder	Nicolas Pinheiro Silva	Função: Líder.
Redator	Karen Helloisa Araújo Silva	Funções: Implementação do circuito Subtrator. Relatório.
Debatedor	Rinaldo Tavares da Silva Filho	Funções: Implementação do circuito Liberador. Relatório.

B.2 Defina o problema

Implementação em vhdl de um maquina de troco.

B.3 Pontos-chaves

Implementação em vhdl, maquina de estados de alto nível;

B.4 Questões de pesquisa

Implementação em vhdl, processing;

B.5 Planejamento da pesquisa

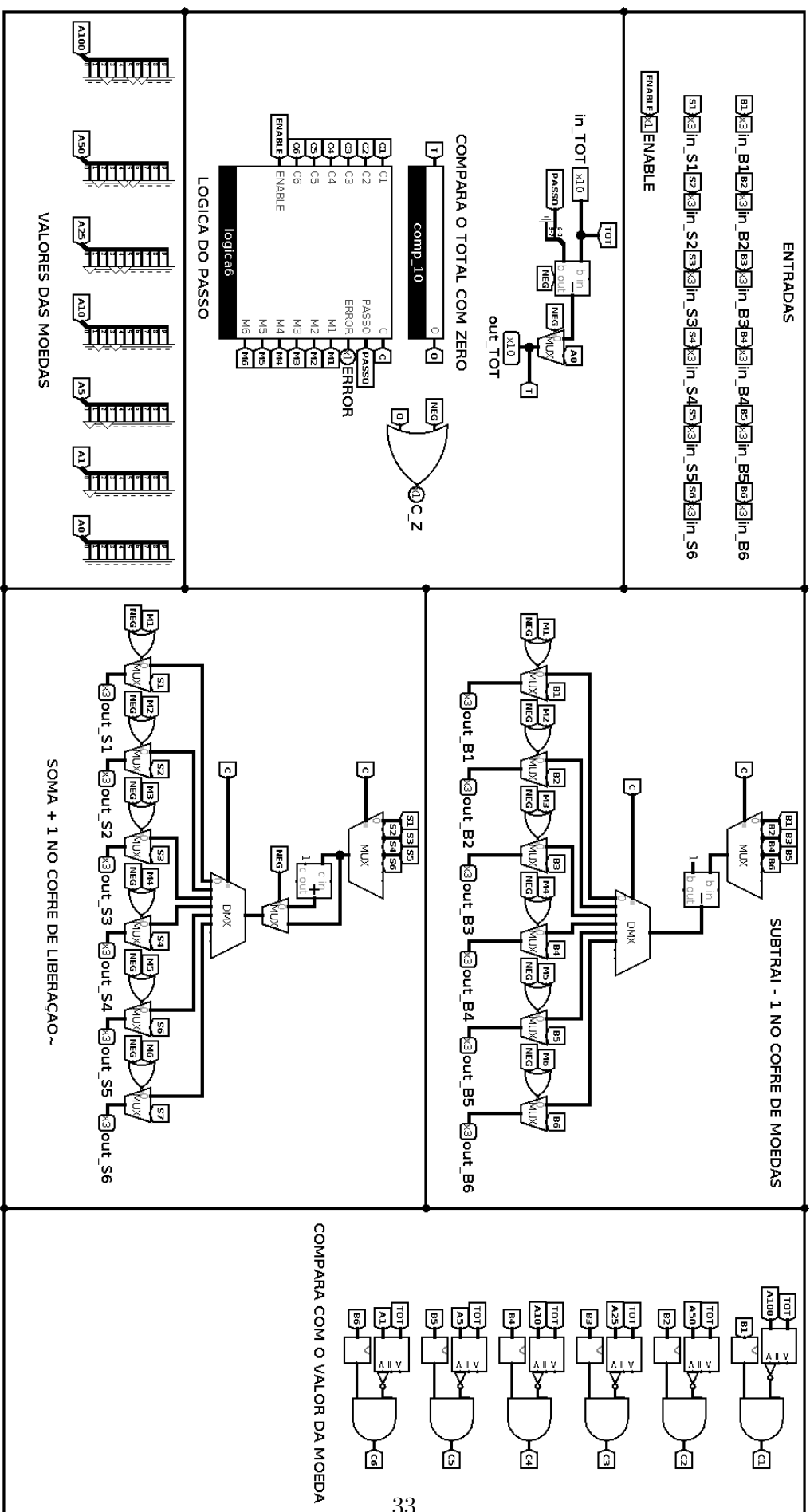
([Vahid 2008](#)) apenas.

C Anexos

LOGICA DO PASSO

Tabela C.1: Equações lógicas da unidade de controle `logica_passo`

Sinal	Equação Lógica
C_2	$\overline{C_1} \overline{C_2} \overline{C_3} \overline{C_4} + \overline{ENABLE}$
C_1	$\overline{C_1} \overline{C_2} \overline{C_5} \overline{C_6} + \overline{ENABLE} + \overline{C_1} \overline{C_2} C_4 + \overline{C_1} \overline{C_2} C_3$
C_0	$\overline{C_1} \overline{C_3} \overline{C_5} + \overline{ENABLE} + \overline{C_1} \overline{C_3} C_4 + \overline{C_1} C_2$
$PASSO_6$	$C_1 \overline{ENABLE}$
$PASSO_5$	$(C_1 + C_2) \overline{ENABLE}$
$PASSO_4$	$(\overline{C_1} C_2 + \overline{C_1} C_3) \overline{ENABLE}$
$PASSO_3$	$(\overline{C_1} \overline{C_2} C_3 + \overline{C_1} \overline{C_2} C_4) \overline{ENABLE}$
$PASSO_2$	$C_1 \overline{ENABLE} + \overline{C_2} \overline{C_3} \overline{C_4} C_5 \overline{ENABLE}$
$PASSO_1$	$(\overline{C_1} C_2 + \overline{C_1} \overline{C_3} C_4) \overline{ENABLE}$
$PASSO_0$	$(\overline{C_1} \overline{C_2} C_3 + \overline{C_1} \overline{C_2} \overline{C_4} C_6 + \overline{C_1} \overline{C_2} \overline{C_4} C_5) \overline{ENABLE}$
E_{error}	$\overline{C_1} \overline{C_2} \overline{C_3} \overline{C_4} \overline{C_5} \overline{C_6} \overline{ENABLE}$
M_1	$\overline{C_1} + \overline{ENABLE}$
M_2	$\overline{C_2} + \overline{ENABLE} + C_1$
M_3	$\overline{C_3} + \overline{ENABLE} + C_2 + C_1$
M_4	$\overline{C_4} + \overline{ENABLE} + C_3 + C_2 + C_1$
M_5	$\overline{C_5} + \overline{ENABLE} + C_4 + C_3 + C_2 + C_1$
M_6	$\overline{C_6} + \overline{ENABLE} + C_5 + C_4 + C_3 + C_2 + C_1$



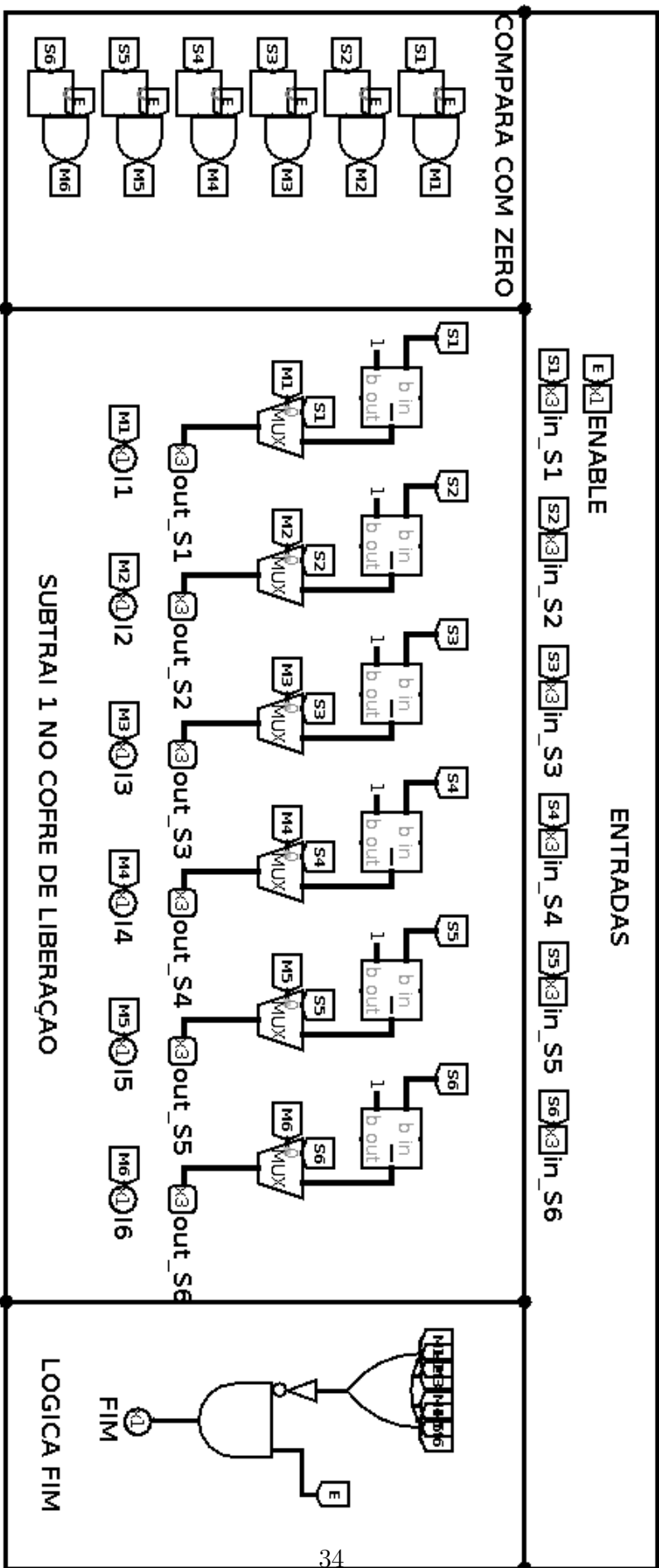


Figura C.2: Circuito Liberador

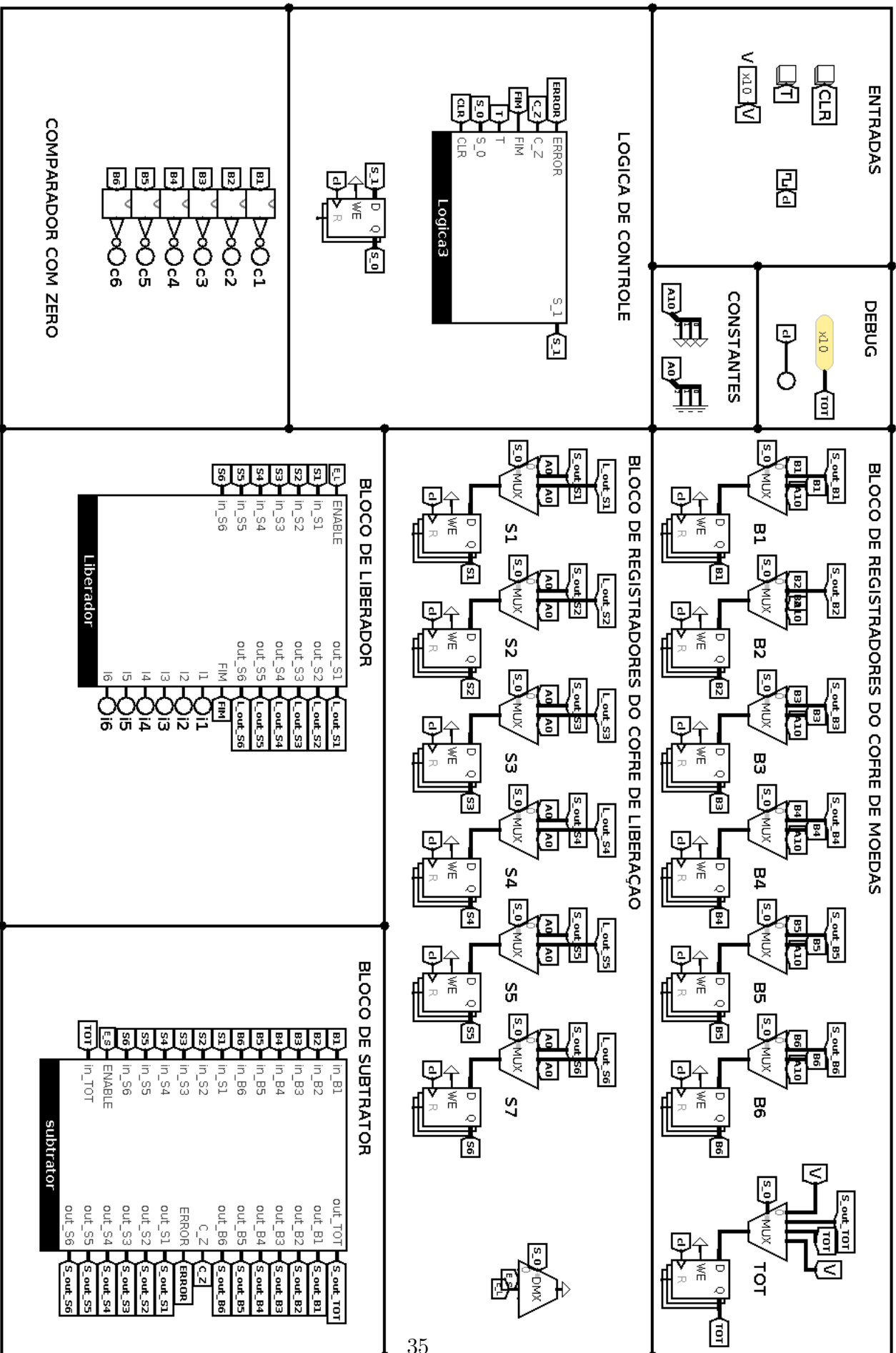


Figura C.3: Máquina de troco